

# **REPORT**

## **ASSIGNMENT 4**

**ANUSHKA GUPTA**                      **220101014**

**KASHISH AGGARWAL**                **220101050**

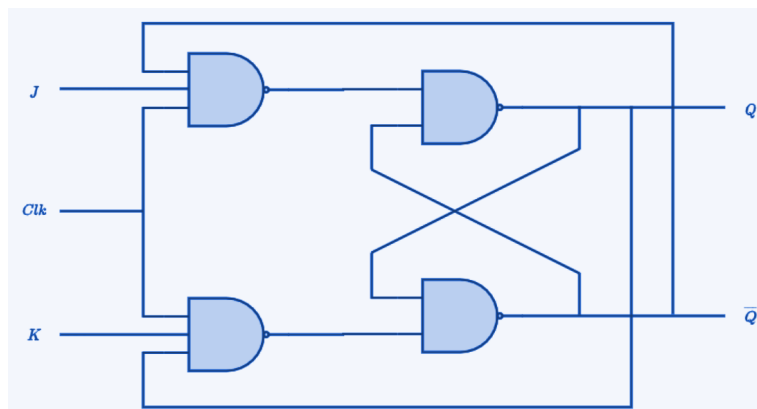
**MAHEK CHOUDHARY**                **220101062**

**MAHUA SINGH**                        **220101063**

# J-K FLIP FLOPS

The JK flip-flop is a gated SR flip-flop with the addition of a clock input circuitry that prevents the illegal or invalid output condition that can occur when both inputs S and R are equal to logic level “1”. Due to this additional clocked input, a JK flip-flop has four possible input combinations, “logic 1”, “logic 0”, “no change” and “toggle”. The symbol for a JK flip flop is similar to that of an SR Bistable Latch as seen in the previous tutorial except for the addition of a clock input.

| J | K | $Q_{(n+1)}$ | State     |
|---|---|-------------|-----------|
| 0 | 0 | $Q_n$       | No Change |
| 0 | 1 | 0           | RESET     |
| 1 | 0 | 1           | SET       |
| 1 | 1 | $Q_n'$      | TOGGLE    |



# **COUNTERS**

A Counter is a device which stores (and sometimes displays) the number of times a particular event or process has occurred, often in relationship to a clock signal. Counters are used in digital electronics for counting purposes, they can count specific events happening in the circuit. For example, in UP counter a counter increases the count for every rising edge of the clock. Not only counting, a counter can follow a certain sequence based on our design like any random sequence 0,1,2 and so on. They can also be designed with the help of flip-flops.

They are used as frequency dividers where the frequency of a given pulse waveform is divided. Counters are sequential circuits that count the number of pulses can be either in binary code or BCD form. The main properties of a counter are timing, sequencing, and counting.

**Counter works in two modes -:**

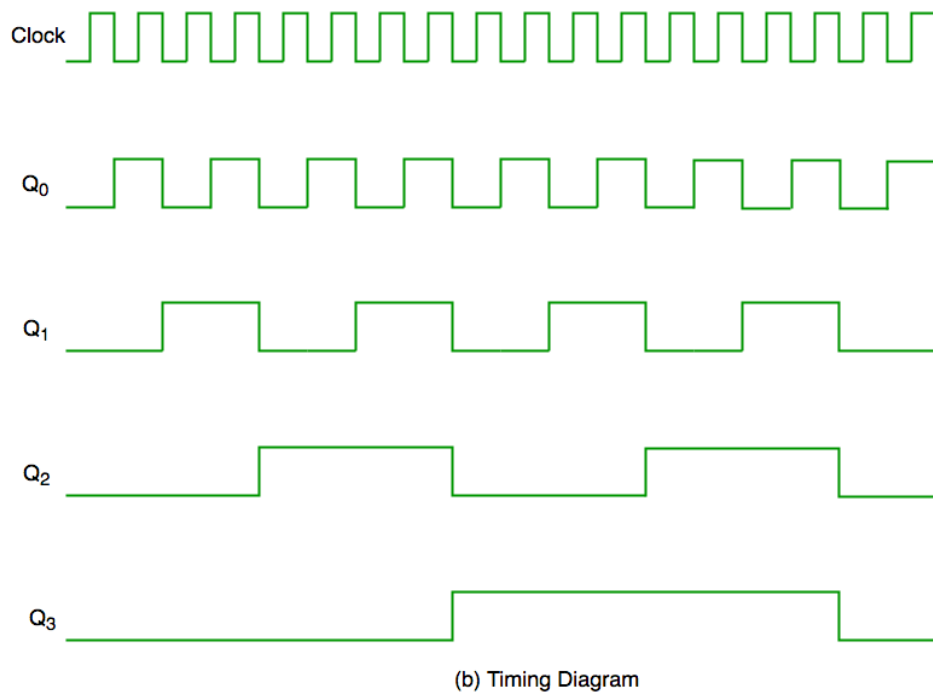
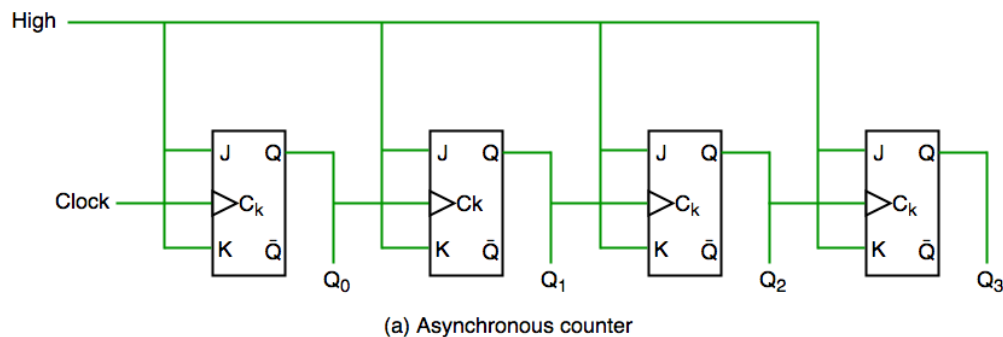
- 1) Up counter
- 2) Down counter

**Counters are classified into two categories as follows -:**

- 1) Asynchronous Counters
- 2) Synchronous Counters

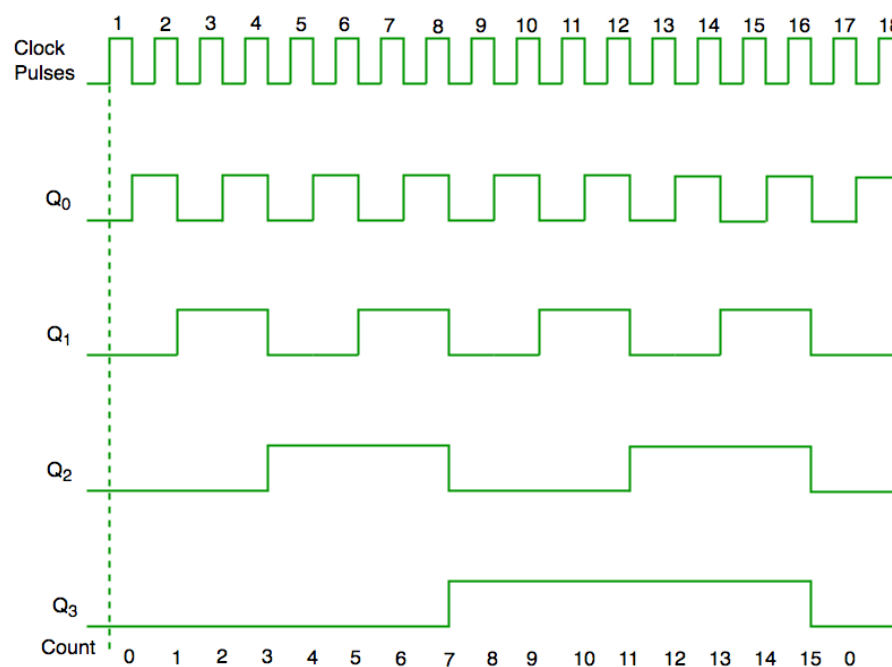
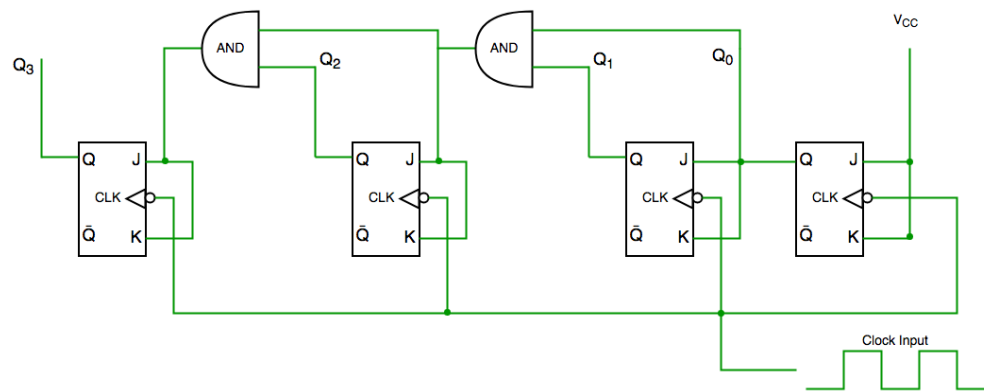
# ASYNCHRONOUS COUNTERS

In the asynchronous counter we don't use a universal clock, only the first flip-flop is driven by the main clock and the clock input of the rest of the following flip-flop is driven by the output of previous flip-flops. We can understand it by following the diagram -:



# SYNCHRONOUS COUNTERS

Unlike the asynchronous counter, synchronous counter has one global clock which drives each flip flop so output changes in parallel. The one advantage of a synchronous counter over an asynchronous counter is, it can operate on a higher frequency than an asynchronous counter as it does not have cumulative delay because of same clock is given to each flip flop. It is also called a parallel counter.



# **4- BIT SYNCHRONOUS**

# **UP/DOWN COUNTER**

The objective is to design a 4-bit synchronous up-down counter capable of counting in both upward and downward directions. Additionally, the counter should incorporate set and reset functionalities to allow for manual initialization and control of its state.

## **Truth Table and Behavioral Analysis:**

We created a truth table with columns for the 4-bit outputs (QA, QB, QC, QD), the control signal 'up', and the set and reset inputs.

Analyze the behaviour of the counter based on the truth table, considering the effects of the 'up', set, and reset inputs on the counter's state transitions.

## **State Transition Logic:**

We derived the logic expressions for the JK inputs of each flip-flop to implement the set and reset functionalities.

Then we determined the logic expressions for the counter's state transitions based on the current state, the 'up' signal, and the set and reset inputs.

## UP = 0 (DOWN COUNTER)

| Q <sub>3</sub> | Q <sub>2</sub> | Q <sub>1</sub> | Q <sub>0</sub> | UP | Q <sub>3</sub> <sup>+</sup> | Q <sub>2</sub> <sup>+</sup> | Q <sub>1</sub> <sup>+</sup> | Q <sub>0</sub> <sup>+</sup> | J <sub>3</sub> | K <sub>3</sub> | J <sub>2</sub> | K <sub>2</sub> | J <sub>1</sub> | K <sub>1</sub> | J <sub>0</sub> | K <sub>0</sub> |
|----------------|----------------|----------------|----------------|----|-----------------------------|-----------------------------|-----------------------------|-----------------------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|
| 0              | 0              | 0              | 0              | 0  | 1                           | 1                           | 1                           | 1                           | 1              | x              | 1              | x              | 1              | x              | 1              | x              |
| 1              | 1              | 1              | 1              | 0  | 1                           | 1                           | 1                           | 0                           | x              | 0              | x              | 0              | x              | 0              | x              | 1              |
| 1              | 1              | 1              | 0              | 0  | 1                           | 1                           | 0                           | 1                           | x              | 0              | x              | 0              | x              | 1              | 1              | x              |
| 1              | 1              | 0              | 1              | 0  | 1                           | 1                           | 0                           | 0                           | x              | 0              | x              | 0              | 0              | x              | x              | 1              |
| 1              | 1              | 0              | 0              | 0  | 1                           | 0                           | 1                           | 1                           | x              | 0              | x              | 1              | 1              | x              | 1              | x              |
| 1              | 0              | 1              | 1              | 0  | 1                           | 0                           | 1                           | 0                           | x              | 0              | 0              | x              | x              | 0              | x              | 1              |
| 1              | 0              | 1              | 0              | 0  | 1                           | 0                           | 0                           | 1                           | x              | 0              | 0              | x              | x              | 1              | 1              | x              |
| 1              | 0              | 0              | 1              | 0  | 1                           | 0                           | 0                           | 0                           | x              | 0              | 0              | x              | 0              | x              | x              | 1              |
| 1              | 0              | 0              | 0              | 0  | 0                           | 1                           | 1                           | 1                           | x              | 1              | 1              | x              | 1              | x              | 1              | x              |
| 0              | 1              | 1              | 1              | 0  | 0                           | 1                           | 1                           | 0                           | 0              | x              | x              | 0              | x              | 0              | x              | 1              |
| 0              | 1              | 1              | 0              | 0  | 0                           | 1                           | 0                           | 1                           | 0              | x              | x              | 0              | x              | 1              | 1              | x              |
| 0              | 1              | 0              | 1              | 0  | 0                           | 1                           | 0                           | 0                           | 0              | x              | x              | 0              | 0              | x              | x              | 1              |
| 0              | 1              | 0              | 0              | 0  | 0                           | 0                           | 1                           | 1                           | 0              | x              | x              | 1              | 1              | x              | 1              | x              |
| 0              | 0              | 1              | 1              | 0  | 0                           | 0                           | 1                           | 0                           | 0              | x              | 0              | x              | x              | 0              | x              | 1              |
| 0              | 0              | 1              | 0              | 0  | 0                           | 0                           | 0                           | 1                           | 0              | x              | 0              | x              | x              | 1              | 1              | x              |
| 0              | 0              | 0              | 1              | 0  | 0                           | 0                           | 0                           | 0                           | 0              | x              | 0              | x              | 0              | x              | x              | 1              |

## UP = 1 (UP COUNTER)

| Q <sub>3</sub> | Q <sub>2</sub> | Q <sub>1</sub> | Q <sub>0</sub> | UP | Q <sub>3</sub> <sup>+</sup> | Q <sub>2</sub> <sup>+</sup> | Q <sub>1</sub> <sup>+</sup> | Q <sub>0</sub> <sup>+</sup> | J <sub>3</sub> | K <sub>3</sub> | J <sub>2</sub> | K <sub>2</sub> | J <sub>1</sub> | K <sub>1</sub> | J <sub>0</sub> | K <sub>0</sub> |
|----------------|----------------|----------------|----------------|----|-----------------------------|-----------------------------|-----------------------------|-----------------------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|
| 0              | 0              | 0              | 0              | 1  | 0                           | 0                           | 0                           | 1                           | 0              | x              | 0              | x              | 0              | x              | 1              | x              |
| 1              | 1              | 1              | 1              | 1  | 0                           | 0                           | 1                           | 0                           | 0              | x              | 0              | x              | 1              | x              | x              | 1              |
| 1              | 1              | 1              | 0              | 1  | 0                           | 0                           | 1                           | 1                           | 0              | x              | 0              | x              | x              | 0              | 1              | x              |
| 1              | 1              | 0              | 1              | 1  | 0                           | 1                           | 0                           | 0                           | 0              | x              | 1              | x              | x              | 1              | x              | 1              |
| 1              | 1              | 0              | 0              | 1  | 0                           | 1                           | 0                           | 1                           | 0              | x              | x              | 0              | 0              | x              | 1              | x              |
| 1              | 0              | 1              | 1              | 1  | 0                           | 1                           | 1                           | 0                           | 0              | x              | x              | 0              | 1              | x              | x              | 1              |
| 1              | 0              | 1              | 0              | 1  | 0                           | 1                           | 1                           | 1                           | 0              | x              | x              | 0              | x              | 0              | 1              | x              |
| 1              | 0              | 0              | 1              | 1  | 1                           | 0                           | 0                           | 0                           | 1              | x              | x              | 1              | x              | 1              | x              | 1              |
| 1              | 0              | 0              | 0              | 1  | 1                           | 0                           | 0                           | 1                           | x              | 0              | 0              | x              | 0              | x              | 1              | x              |
| 0              | 1              | 1              | 1              | 1  | 1                           | 0                           | 1                           | 0                           | x              | 0              | 0              | x              | 1              | x              | x              | 1              |
| 0              | 1              | 1              | 0              | 1  | 1                           | 0                           | 1                           | 1                           | x              | 0              | 0              | x              | x              | 0              | 1              | x              |
| 0              | 1              | 0              | 1              | 1  | 1                           | 1                           | 0                           | 0                           | x              | 0              | 1              | x              | x              | 1              | x              | 1              |
| 0              | 1              | 0              | 0              | 1  | 1                           | 1                           | 0                           | 1                           | x              | 0              | x              | 0              | 0              | x              | 1              | x              |
| 0              | 0              | 1              | 1              | 1  | 1                           | 1                           | 1                           | 0                           | x              | 0              | x              | 0              | 1              | x              | x              | 1              |
| 0              | 0              | 1              | 0              | 1  | 1                           | 1                           | 1                           | 1                           | x              | 0              | x              | 0              | x              | 0              | 1              | x              |
| 0              | 0              | 0              | 1              | 1  | 0                           | 0                           | 0                           | 0                           | x              | 1              | x              | 1              | x              | 1              | x              | 1              |



$$J_0 = 1$$

$$J_1 = Q_0(\text{up}) + Q_0'(\text{up})'$$

$$J_2 = Q_1Q_0(\text{up}) + Q_1'Q_0'(\text{up})'$$

$$J_3 = Q_2Q_1Q_0(\text{up}) + Q_2'Q_1'Q_0'(\text{up})'$$

$$K_0 = 1$$

$$K_1 = Q_0(\text{up}) + Q_0'(\text{up})'$$

$$K_2 = Q_1Q_0(\text{up}) + Q_1'Q_0'(\text{up})'$$

$$K_3 = Q_2Q_1Q_0(\text{up}) + Q_2'Q_1'Q_0'(\text{up})'$$



Logic  
Expressions

### Circuit Design:

We implemented the derived logic expressions using JK flip-flops.

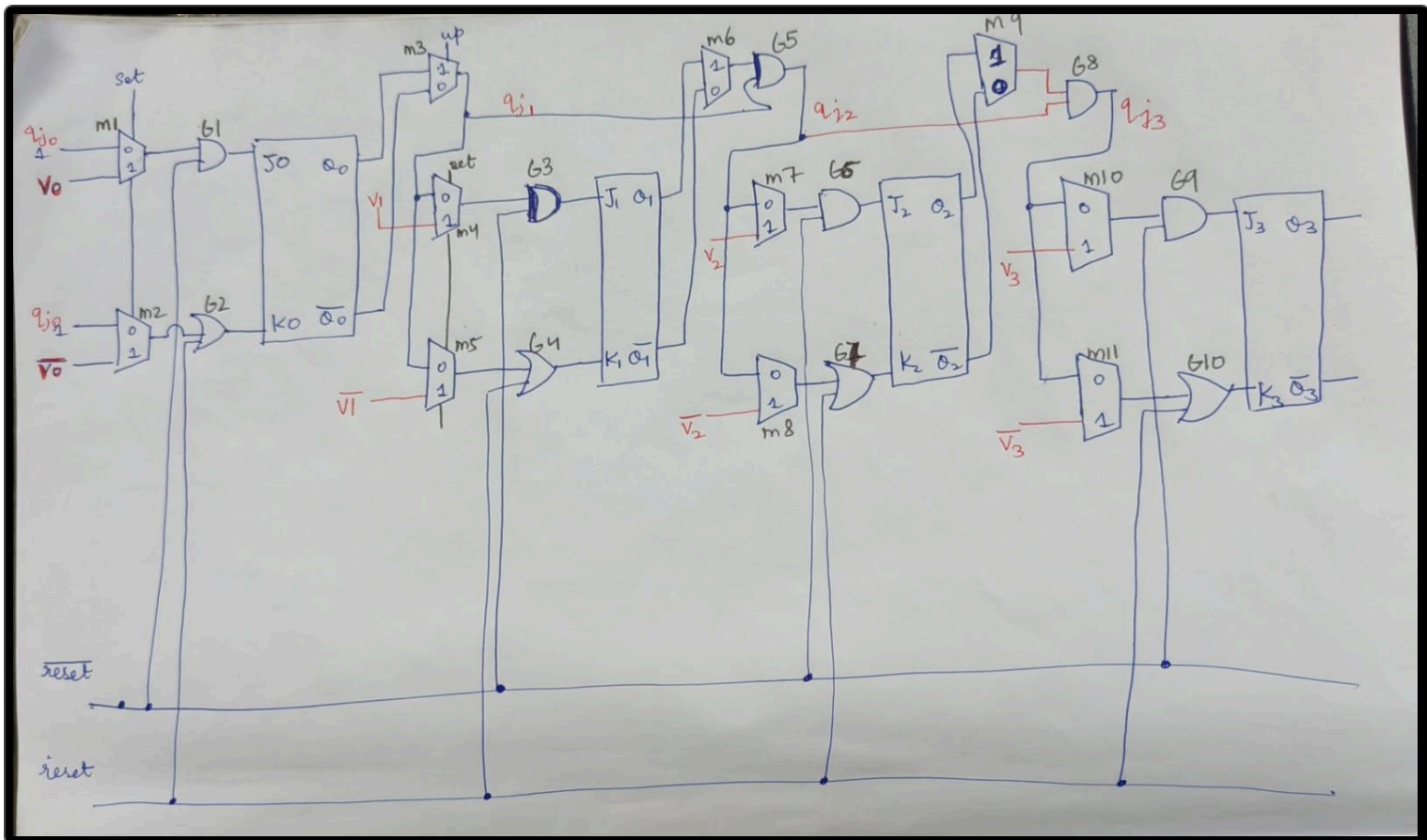
We connected the set and reset inputs to the appropriate JK inputs of the flip-flops, ensuring proper synchronization with the clock signal.

### Testing and Simulation:

We simulated the circuit using a digital logic simulator to verify its functionality.

Then we tested the counter under various conditions, including different initial states, 'up' signal values, and set/reset input combinations, to ensure correct operation.

## CIRCUIT DIAGRAM



# ICs USED IN IMPLEMENTATION

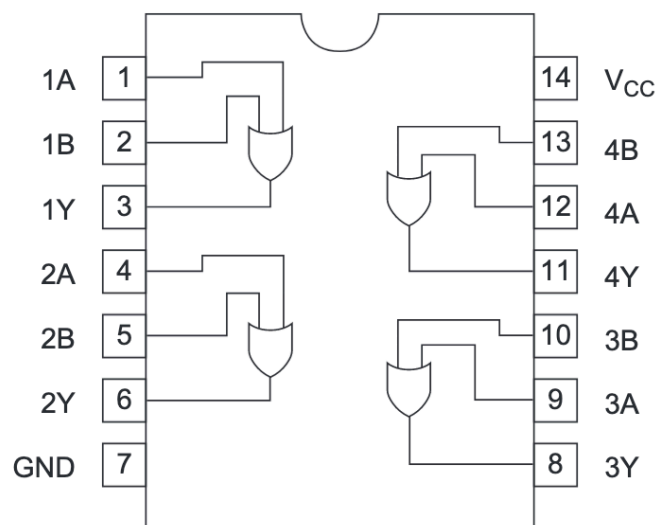
## HD74LS32

### (Quadruple 2-input Positive OR Gate)

The HD74LS32 is a quad 2-input OR gate integrated circuit (IC) chip belonging to the 74LS family of TTL (Transistor-Transistor Logic) devices. It contains four individual OR gates, each with two inputs and one output.

#### Functionality:

The primary function of an OR gate is to output a high signal (logic 1) if any of its input signals are high (logic 1). Otherwise, it outputs a low signal (logic 0). With four OR gates in the HD74LS32, it can perform logical OR operations on multiple pairs of binary inputs simultaneously.



(Top view)

# SN74LS157

## Quadruple 2 : 1 Selector / Multiplexer

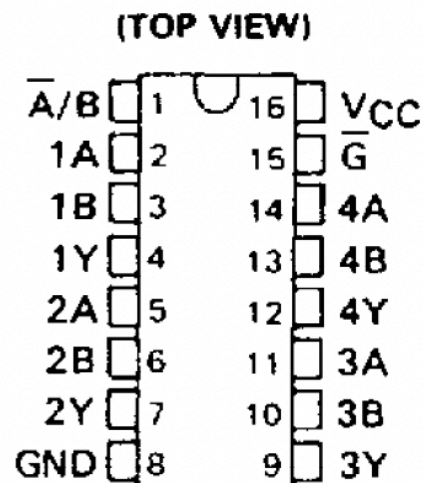
The SN74LS157 is a widely used integrated circuit (IC) chip that belongs to the 74LS series of TTL (Transistor-Transistor Logic) devices. It is specifically a quad 2-input multiplexer, meaning it contains four individual multiplexers within a single package.

### Functionality:

The primary function of the SN74LS157 is to perform multiplexing operations. Each of the four multiplexers within the chip selects one of two input data lines based on a select input. The selected data line is then passed through to the output.

### Application:

The SN74LS157 is commonly used in digital circuits for data selection, signal routing, and addressing purposes. It's often employed in multiplexing and demultiplexing operations, as well as in various digital systems requiring the selection of multiple input signals based on a control input.



# HD74LS04

## Hex Inverters / NOT Gates

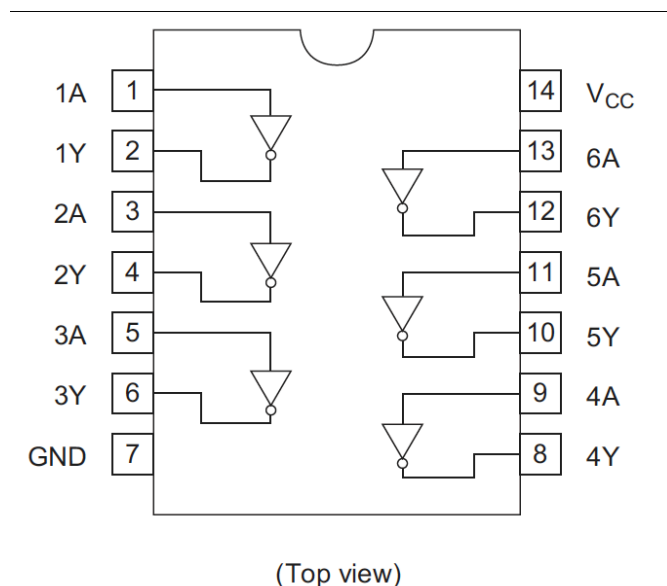
The HD74LS04 is a popular integrated circuit (IC) chip that belongs to the 74LS series of TTL (Transistor-Transistor Logic) devices. It is specifically a hex inverter, meaning it contains six individual inverters within a single package.

### Functionality:

The primary function of the HD74LS04 is to perform inversion operations. Each of the six inverters within the chip takes an input signal and produces the complementary output (i.e., it outputs the logical opposite of the input).

### Applications:

The HD74LS04 is commonly used in digital circuits for signal inversion, level shifting, and signal buffering. It's often employed to convert logic levels or to complement the output of logic gates, such as AND, OR, and NAND gates.



# HD74LS08

## Quadruple 2-Input Positive AND Gates

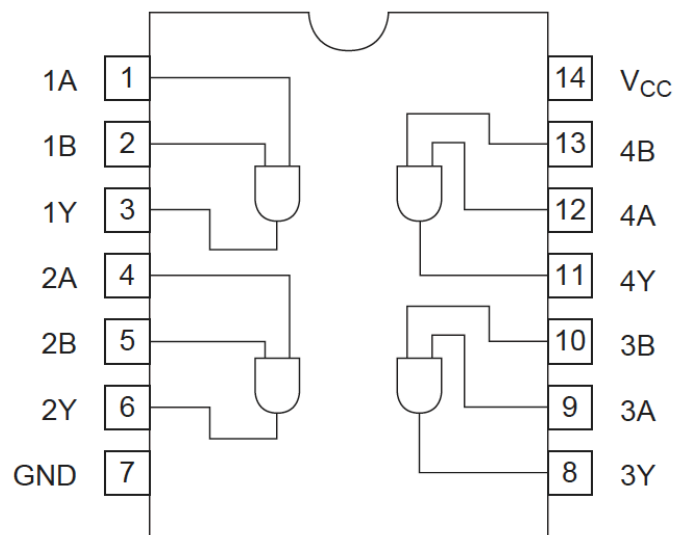
The HD74LS08 is an integrated circuit (IC) chip belonging to the 74LS series of TTL (Transistor-Transistor Logic) devices. It is specifically a quad 2-input AND gate, containing four individual AND gates within a single package.

### Functionality:

The primary function of the HD74LS08 is to perform logical AND operations. Each of the four AND gates within the chip accepts two binary inputs and produces a single binary output based on the logical AND operation.

### Applications:

The HD74LS08 is commonly used in digital circuits for performing logical AND operations. It's often employed in conjunction with other logic gates, such as OR gates and NOT gates, to implement various Boolean functions and to control signal flow in digital systems.



(Top view)

# SN74LS76A

## DUAL J-K FLIP-FLOPS (PRESET AND CLEAR)

The SN74LS76A is a dual JK flip-flop IC that belongs to the 74LS series of TTL (Transistor-Transistor Logic) devices. It contains two independent JK flip-flops within a single package.

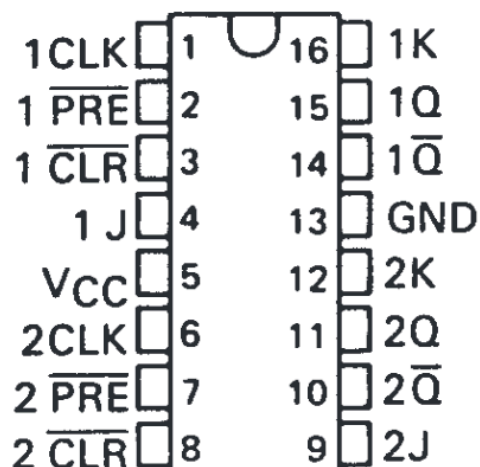
### Functionality:

The SN74LS76A is designed to store binary data and manipulate it based on clock and control inputs. Each JK flip-flop within the chip has J (set), K (reset), CLK (clock), and CLR (clear) inputs, as well as Q (normal output) and Q' (complementary output) outputs.

### Applications:

The SN74LS76A is commonly used in digital circuits for storing and manipulating binary data, implementing sequential logic functions, and controlling the timing of operations. It's often employed in counters, registers, and other sequential logic circuits where the storage and manipulation of binary data are required.

(TOP VIEW)

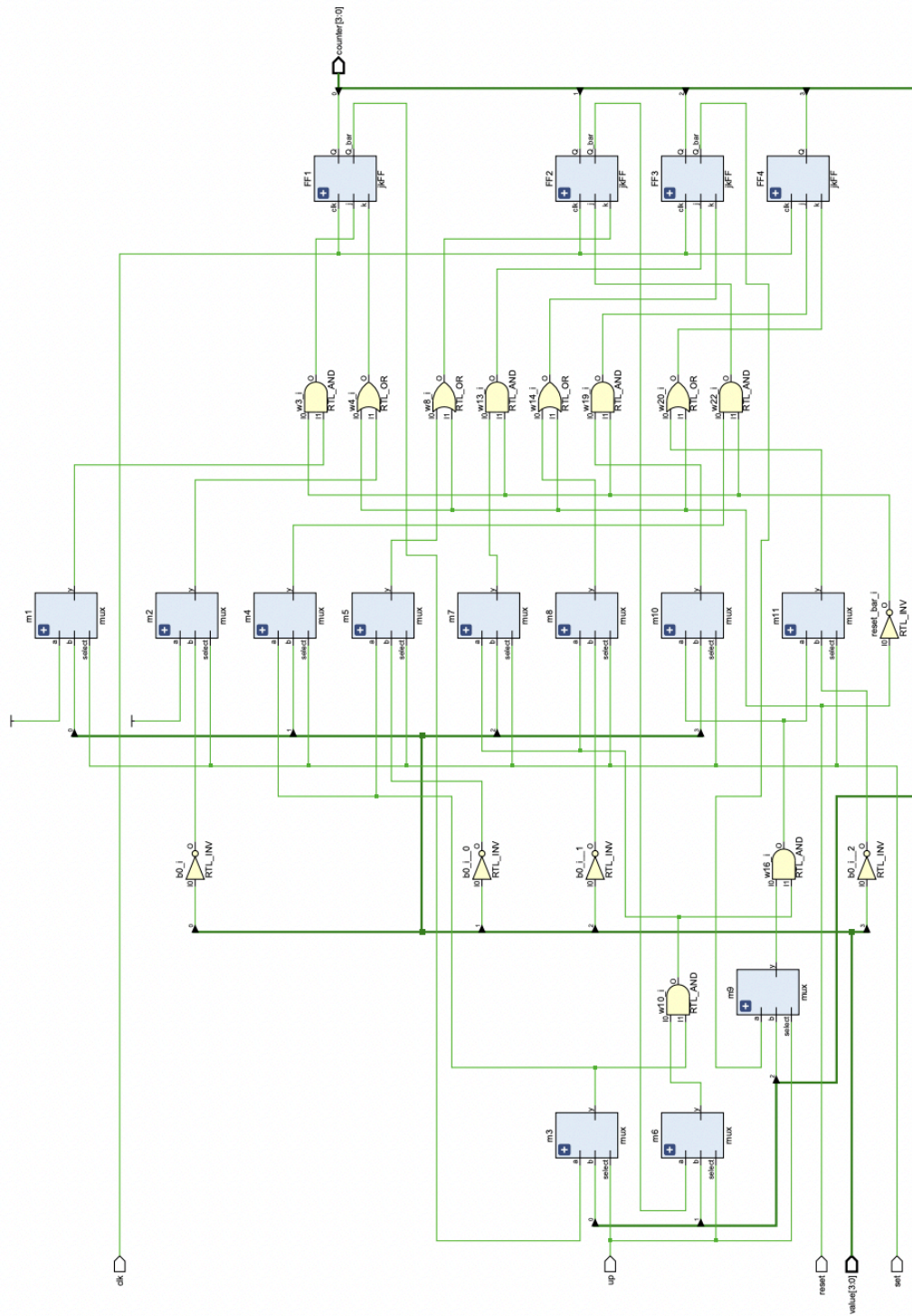


## TEST SIMULATIONS





# CIRCUIT SCHEMATIC



# WORKING CIRCUIT ON BREADBOARD

